

Technical Documentation: Integrated Numerical Interpolation Suite (GUI Edition)

Group: 3

Team Lead: Daksh Agarwal
Team Member 1: Gugulothu Maniprakash
Team Member 2: Maneesh S Kolekar
Team Member 3: Vaibhav Singh
Team Member 4: Samudrala Manish Kumar

Version: 1.0 | Date: February 14, 2026

Abstract

This document serves as the technical specification for Phase 3 of the Integrated Interpolation Project. The system has evolved from a console application into a modular Windows Desktop Application. It features a user-friendly Graphical User Interface (GUI) developed in Visual Basic .NET (Windows Forms) and a high-performance backend library developed in C++. The system ensures seamless interoperability via Dynamic Link Libraries (DLLs) and includes a professional MSI installer for deployment.

Contents

1	System Overview	3
1.1	Purpose and Scope	3
1.2	System Architecture	3
2	Component Specifications	3
2.1	GUI Controls (Visual Basic)	3
2.2	Backend Functions (C++ DLL)	3
3	Interoperability & Integration	4
3.1	DLL Export Mechanism (C++)	4
3.2	DLL Import Mechanism (Visual Basic)	4
4	Algorithm Implementation Details	4
4.1	Method 1: Piecewise Constant Interpolation	4
4.2	Method 2: Nearest Neighbor Interpolation	4
4.3	Method 3: Linear Interpolation	4
4.4	Method 4: Newton's Divided Difference	4
4.5	Method 5: Lagrange Interpolation	5
5	Event Handling Mechanism	5
5.1	Calculation Workflow (btnCalculate_Click)	5

6	Deployment Strategy	5
6.1	MSI Installer Generation	5
6.2	Dependencies	5
7	Execution Guide	5
7.1	Installation	5
7.2	Runtime Instructions	6

1 System Overview

1.1 Purpose and Scope

The objective of Phase 3 is to democratize access to numerical analysis tools by replacing the command-line interface with an intuitive GUI. The application allows users to perform interpolation (Piecewise, Nearest Neighbor, Linear, Newton, Lagrange) via a grid-based input system, with real-time validation and error handling.

1.2 System Architecture

The software follows a **Multi-Tier Architecture**, separating the presentation layer from the business logic layer.

1. **Frontend (Presentation Layer):** Developed in **Visual Basic .NET**. Responsible for User Interface (UI), input collection, data validation, and result rendering.
2. **Interop Layer (The Bridge):** Utilizes **P/Invoke (Platform Invoke)** to facilitate communication between the managed .NET environment and unmanaged C++ memory space.
3. **Backend (Computation Layer):** Developed in **Native C++**. Compiled as a Dynamic Link Library (`InterpolationBackend.dll`) containing the core mathematical algorithms.

2 Component Specifications

2.1 GUI Controls (Visual Basic)

The user interface is designed using standard Windows Forms controls:

- **dgvData (DataGridView):** A dynamic table allowing users to enter N rows of (x, y) coordinate pairs.
- **txtTarget (TextBox):** Accepts the target X value for interpolation.
- **cmbMethod (ComboBox):** A dropdown list to select one of the five interpolation algorithms.
- **btnCalculate (Button):** The trigger for the validation and calculation events.
- **lblResult (Label):** Displays the calculated output or status messages.

2.2 Backend Functions (C++ DLL)

The backend exports the following functions using the C-calling convention ($_{cdecl}$):

`interpolatePiecewise`

`interpolateNearestNeighbor`

`interpolateLinear`

`interpolateNewton`

`interpolateLagrange`

3 Interoperability & Integration

3.1 DLL Export Mechanism (C++)

To make C++ functions accessible to Visual Basic, we prevent name mangling and export the symbols explicitly.

```
1 #define EXPORT extern "C" __declspec(dllexport)
2
3 EXPORT double interpolateLinear(double targetX, double* x, double* y, int n
4     ) {
5     // Logic implementation...
```

Listing 1: C++ Export Macro

3.2 DLL Import Mechanism (Visual Basic)

The frontend declares references to the external DLL functions.

```
1 <DllImport("InterpolationBackend.dll", CallingConvention:=CallingConvention
2     .Cdecl)>
3 Public Shared Function interpolateLinear( _
4     ByVal targetX As Double, _
5     ByVal x() As Double, _
6     ByVal y() As Double, _
7     ByVal n As Integer _
8 ) As Double
9 End Function
```

Listing 2: Visual Basic DllImport Declaration

4 Algorithm Implementation Details

4.1 Method 1: Piecewise Constant Interpolation

Assumes the function value remains constant between data points.

$$f(x) = y_i \quad \text{for} \quad x_i \leq x < x_{i+1}$$

4.2 Method 2: Nearest Neighbor Interpolation

Selects the value of the data point geometrically closest to the target x .

$$f(x) = y_k \quad \text{where} \quad |x - x_k| = \min_j |x - x_j|$$

4.3 Method 3: Linear Interpolation

Estimates value by connecting adjacent points with a straight line.

$$f(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i)$$

4.4 Method 4: Newton's Divided Difference

Constructs a polynomial using recursive difference tables.

$$P(x) = f[x_0] + (x - x_0)f[x_0, x_1] + \dots$$

4.5 Method 5: Lagrange Interpolation

A linear combination of basis polynomials $L_i(x)$.

$$P(x) = \sum_{i=0}^n y_i L_i(x) \quad \text{where} \quad L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

5 Event Handling Mechanism

The application relies on an Event-Driven architecture managed by the VB.NET runtime.

5.1 Calculation Workflow (`btnCalculate_Click`)

1. **Data Extraction:** Reads N value and resizes internal arrays. Loops through `dgvData` rows to populate X and Y arrays.
2. **Validation Layer:**
 - **Type Checking:** Uses `Double.TryParse` to ensure all inputs are numeric.
 - **Constraint Checking:** Scans for duplicate X values (preventing division by zero).
 - **Sorting:** Automatically sorts arrays by X value to ensure algorithm correctness.
 - **Range Checking:** Prevents extrapolation for Piecewise/Linear methods if $Target < X_{min}$ or $Target > X_{max}$.
3. **Backend Invocation:** Calls the specific imported function based on `cmbMethod.SelectedIndex`.
4. **Result Rendering:** Formats the returned double to 4 decimal places and updates `txtResult`.

6 Deployment Strategy

6.1 MSI Installer Generation

The software is packaged using a **Visual Studio Setup Project**.

- **Project Output Group:** Bundles `InterpolationGUI.exe` (Primary Output) and `InterpolationBackend.dll` (Content File).
- **File System Management:** Installs files to `Program Files \CS346 Group 3 \Interpolation Suite`.
- **Shortcuts:** Automatically creates shortcuts on the User Desktop and Programs Menu.

6.2 Dependencies

The installer bootstrap (`setup.exe`) checks for and installs required components:

- Microsoft .NET Framework 4.0 Client Profile.
- Microsoft Visual C++ 2010 Redistributable Package (x86).

7 Execution Guide

7.1 Installation

1. Navigate to the distribution folder.
2. Run `setup.exe` (Recommended) or `InterpolationSuite.msi`.
3. Follow the wizard instructions to complete installation.

7.2 Runtime Instructions

1. Launch `Interpolation Suite` from the Desktop shortcut. 2. Enter the number of points in the "N" text box. 3. Fill the grid with (x, y) data points. 4. Select a method from the dropdown menu. 5. Enter a target X value and click **Calculate**.